

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Desarrollo de Aplicaciones Web Client and Backend Development Frameworks

Integración de Swagger/OpenAPI

Sistema de Gestión de Cuartos y Reservas

Profesor: M. en C. José Asunción Enríquez Zárate

*Alumna: Estefany Karina Sánchez Calderón
estefanysanchez1495@gmail.com
Grupo: 7CM3*

Índice

1. Objetivo de la práctica	2
2. Descripción del sistema	2
3. Configuración de Swagger/OpenAPI	2
4. Documentación de endpoints	3
4.1. Módulo de cuartos	3
4.2. Módulo de reservaciones	4
5. Seguridad y acceso a la documentación	5
6. Validación de OpenAPI	6
7. Pruebas de funcionamiento	6
8. Conclusiones	7
9. Referencias bibliográficas	8

1 Objetivo de la práctica

El objetivo de la práctica fue integrar Swagger/OpenAPI en el Back End del Sistema de Gestión de Cuartos y Reservaciones, desarrollado con Spring Boot, para generar documentación automática de la API REST y facilitar la prueba de sus endpoints desde una interfaz interactiva.

La integración se enfocó en documentar los módulos principales solicitados por la actividad:

- Habitaciones o cuartos.
- Reservaciones.
- Autenticación y seguridad.

2 Descripción del sistema

El sistema permite administrar cuartos de hotel y registrar reservaciones asociadas a usuarios autenticados. El Back End expone servicios REST bajo el prefijo `/api/v1` y utiliza Spring Security con autenticación HTTP Basic para proteger las operaciones según el rol del usuario.

Módulo	Responsabilidad
Cuarto	Crear, consultar, actualizar, cambiar disponibilidad y eliminar habitaciones.
Reservacion	Crear, consultar, actualizar, cancelar y eliminar reservaciones, validando fechas y traslapes.
Auth	Consultar la sesión del usuario autenticado mediante Spring Security.
Archivo	Cargar y descargar archivos asociados al sistema.

3 Configuración de Swagger/OpenAPI

La documentación automática se habilitó mediante la dependencia `springdoc-openapi-starter-webmvc-ui`. Esta librería genera la especificación OpenAPI en formato JSON y publica Swagger UI para visualizar y probar los endpoints.

Listing 1: Dependencia usada para integrar Springdoc OpenAPI.

```
1 <dependency>
2   <groupId>org.springdoc</groupId>
3   <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
4   <version>2.8.13</version>
5 </dependency>
```

En el archivo `application.properties` se configuraron las rutas de documentación:

Listing 2: Rutas de Swagger UI y OpenAPI JSON.

```
1 springdoc.swagger-ui.path=/swagger-ui.html
2 springdoc.api-docs.path=/v3/api-docs
3 springdoc.swagger-ui.operationsSorter=method
4 springdoc.swagger-ui.tagsSorter=alpha
```

La clase `ApiConfig` define los metadatos principales de la documentación:

Listing 3: Configuración general de OpenAPI.

```
1 @Configuration
2 public class ApiConfig {
3     @Bean
4     public OpenAPI myOpenAPI() {
5         return new OpenAPI()
6             .info(new Info()
7                 .title("API para la gestion de reservaciones")
8                 .version("1.0")
9                 .description("API REST para la gestion de Cuartos y Reservaciones"));
10    }
11 }
```

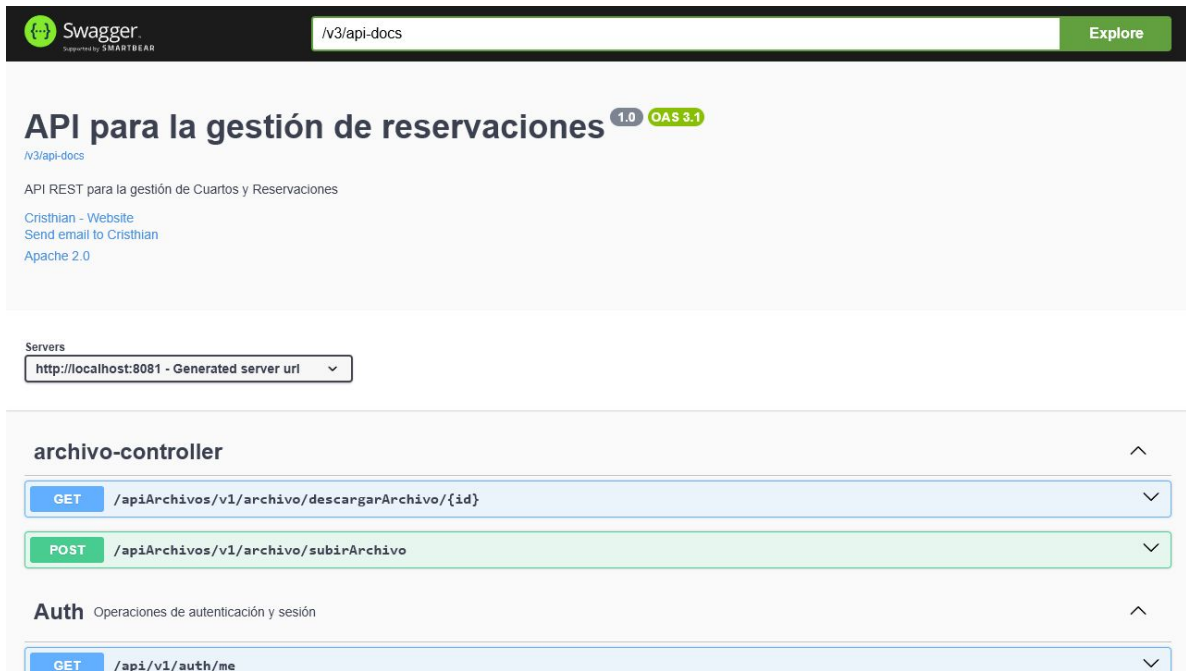


Figura 1: Interfaz Swagger UI del Sistema de Gestión de Cuartos y Reservaciones.

4 Documentación de endpoints

Los controladores principales usan anotaciones de OpenAPI mediante `etag`, lo que permite agrupar las operaciones por módulo dentro de Swagger UI.

4.1 Módulo de cuartos

El módulo `cuarto` se encuentra bajo la ruta `/api/v1/cuartos`. Sus operaciones permiten administrar habitaciones y consultar disponibilidad.

Método	Ruta	Uso
POST	<code>/api/v1/cuartos</code>	Crear cuarto.
GET	<code>/api/v1/cuartos</code>	Listar cuartos.
GET	<code>/api/v1/cuartos/disponibles</code>	Listar cuartos disponibles.
GET	<code>/api/v1/cuartos/{id}</code>	Consultar cuarto por identificador.
PUT	<code>/api/v1/cuartos/{id}</code>	Actualizar datos de un cuarto.
PATCH	<code>/api/v1/cuartos/{id}/disponibilidad</code>	Cambiar disponibilidad.
DELETE	<code>/api/v1/cuartos/{id}</code>	Eliminar cuarto.

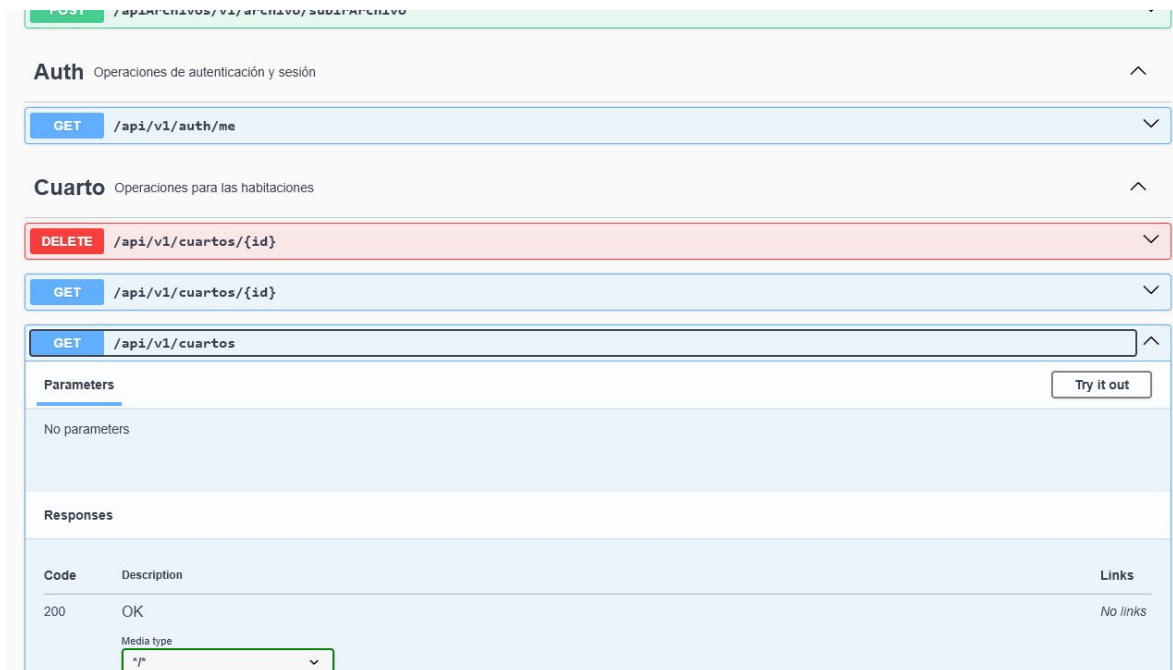


Figura 2: Endpoints del módulo de cuartos documentados en Swagger UI.

4.2 Módulo de reservaciones

El módulo `Reservacion` se encuentra bajo la ruta `/api/v1/reservaciones`. Incluye validaciones para evitar fechas inválidas y traslapes de reservaciones activas sobre el mismo cuarto.

Método	Ruta	Uso
POST	<code>/api/v1/reservaciones</code>	Crear reservación.
GET	<code>/api/v1/reservaciones</code>	Listar todas las reservaciones.
GET	<code>/api/v1/reservaciones/{id}</code>	Consultar reservación por identificador.
GET	<code>/api/v1/reservaciones/usuario/{idUserario}</code>	Consultar reservaciones por usuario.
GET	<code>/api/v1/reservaciones/cuarto/{idCuarto}</code>	Consultar reservaciones por cuarto.
PUT	<code>/api/v1/reservaciones/{id}</code>	Actualizar fechas.
PATCH	<code>/api/v1/reservaciones/{id}/cancelar</code>	Cancelar reservación.
DELETE	<code>/api/v1/reservaciones/{id}</code>	Eliminar reservación.

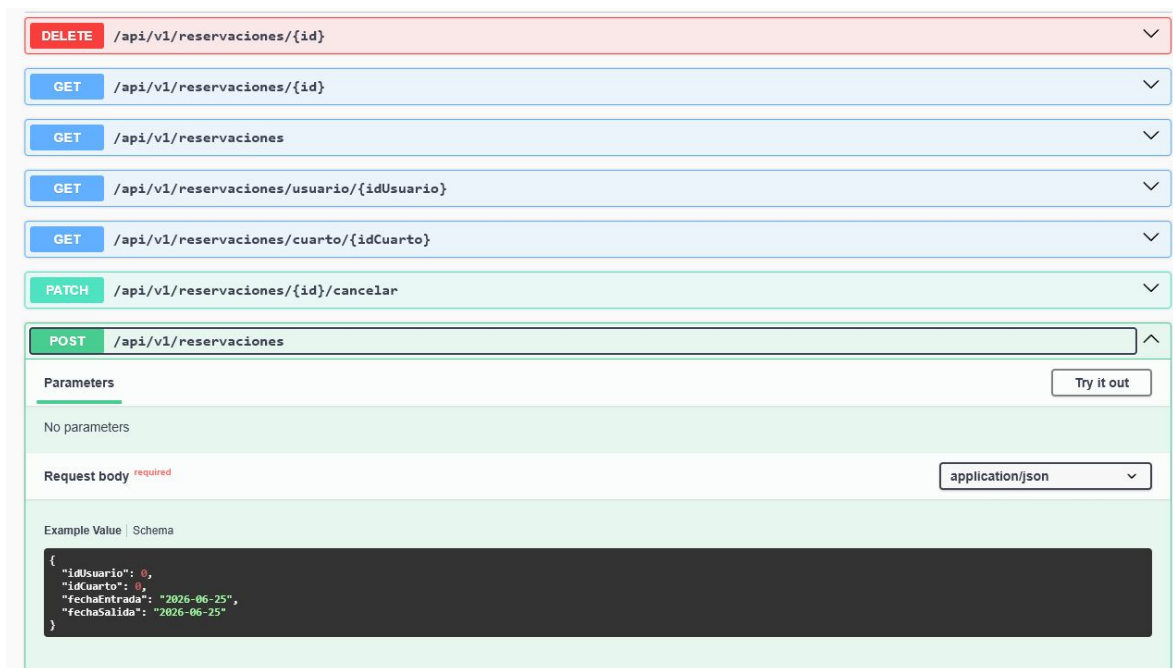


Figura 3: Endpoints del módulo de reservaciones documentados en Swagger UI.

5 Seguridad y acceso a la documentación

La API utiliza Spring Security con HTTP Basic. Los usuarios iniciales se crean desde `DataInitializer`:

Usuario	Contraseña	Rol
admin	123456	ROLE_ADMIN
user	123456	ROLE_USER

La configuración de seguridad permite consultar Swagger UI y OpenAPI JSON sin autenticación, mientras que los endpoints funcionales quedan protegidos por roles.

Listing 4: Reglas principales de seguridad.

```

1 .requestMatchers("/swagger-ui/**", "/v3/api-docs/**", "/swagger-ui.html").permitAll()
2 .requestMatchers(HttpMethod.GET, "/api/v1/cuartos/**").hasAnyRole("ADMIN", "USER")
3 .requestMatchers(HttpMethod.POST, "/api/v1/cuartos/**").hasRole("ADMIN")
4 .requestMatchers(HttpMethod.GET, "/api/v1/reservaciones").hasRole("ADMIN")
5 .requestMatchers(HttpMethod.POST, "/api/v1/reservaciones/**").hasAnyRole("ADMIN", "
  USER")
6 .httpBasic(Customizer.withDefaults());

```

SecurityConfig - Swagger publico y Basic Auth

```
.requestMatchers("/swagger-ui/**", "/v3/api-docs/**", "/swagger-ui.html").permitAll()
.requestMatchers(HttpMethod.GET, "/api/v1/cuartos/**").hasAnyRole("ADMIN", "USER")
.requestMatchers(HttpMethod.POST, "/api/v1/cuartos/**").hasRole("ADMIN")
.requestMatchers(HttpMethod.PUT, "/api/v1/cuartos/**").hasRole("ADMIN")
.requestMatchers(HttpMethod.PATCH, "/api/v1/cuartos/**").hasRole("ADMIN")
.requestMatchers(HttpMethod.DELETE, "/api/v1/cuartos/**").hasRole("ADMIN")
.requestMatchers(HttpMethod.GET, "/api/v1/reservaciones").hasRole("ADMIN")
.requestMatchers(HttpMethod.GET, "/api/v1/reservaciones/cuarto/**").hasRole("ADMIN")
.requestMatchers(HttpMethod.GET, "/api/v1/reservaciones/usuario/**").hasAnyRole("ADMIN", "USER")
.requestMatchers(HttpMethod.GET, "/api/v1/reservaciones/**").hasAnyRole("ADMIN", "USER")
.requestMatchers(HttpMethod.POST, "/api/v1/reservaciones/**").hasAnyRole("ADMIN", "USER")
.requestMatchers(HttpMethod.PUT, "/api/v1/reservaciones/**").hasAnyRole("ADMIN", "USER")
.requestMatchers(HttpMethod.PATCH, "/api/v1/reservaciones/**").hasAnyRole("ADMIN", "USER")
.requestMatchers(HttpMethod.DELETE, "/api/v1/reservaciones/**").hasRole("ADMIN")
.httpBasic(Customizer.withDefaults());
```

Figura 4: Configuración de seguridad para Swagger, cuartos y reservaciones.

6 Validación de OpenAPI

La especificación OpenAPI se puede consultar desde:

<http://localhost:8081/v3/api-docs>

Este documento JSON contiene la versión de OpenAPI, los datos generales de la API, las rutas disponibles, los esquemas de entrada y salida, los parámetros y los códigos de respuesta.

<http://localhost:8081/v3/api-docs>

```
{
  "openapi": "3.1.0",
  "info": {
    "title": "API para la gestión de reservaciones",
    "description": "API REST para la gestión de Cuartos y Reservaciones",
    "contact": {
      "name": "Cristhian",
      "url": "https://cristhanatz@gmail.com",
      "email": "cristhanatz@gmail.com"
    },
    "license": {
      "name": "Apache 2.0",
      "url": "El url de la licencia"
    },
    "version": "1.0"
  },
  "servers": [
    {
      "url": "http://localhost:8081",
      "description": "Generated server url"
    }
  ],
  "tags": [
    {
      "name": "Cuarto",
      "description": "Operaciones para las habitaciones"
    },
    {
      "name": "Reservacion",
      "description": "Operaciones para la gestión de reservaciones"
    },
    {
      "name": "Auth",
      "description": "Operaciones de autenticación y sesión"
    }
  ],
  "paths": {
    "/api/v1/reservaciones/{id}": {
      "get": {
        "description": "Obtener una reservación por ID"
      },
      "post": {
        "description": "Crear una reservación"
      },
      "put": {
        "description": "Actualizar una reservación"
      },
      "delete": {
        "description": "Eliminar una reservación"
      }
    }
  }
}
```

Figura 5: Especificación OpenAPI JSON generada automáticamente.

7 Pruebas de funcionamiento

Se compiló el proyecto con Maven Wrapper usando el comando:

Listing 5: Compilación del proyecto.

```
1 ./mvnw -DskipTests package
```

El resultado fue BUILD SUCCESS, confirmando que la integración de Swagger/OpenAPI no rompe la compilación del Back End.

```
PowerShell - Maven build verification

[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.example:Ejercicio08-Reservaciones >-----
[INFO] Building Ejercicio08-Reservaciones 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ Ejercicio08-Reservaciones ---
[INFO] Copying 2 resources from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.14.1:compile (default-compile) @ Ejercicio08-Reservaciones ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ Ejercicio08-Reservaciones ---
[INFO] skip non existing resourceDirectory C:\Users\luism\Documents\Miguel\Nose\Otro\backend-ejercicio08-main\backend-ejercicio08-main\src\tes
[INFO]
[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ Ejercicio08-Reservaciones ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.5.5:test (default-test) @ Ejercicio08-Reservaciones ---
[INFO] Tests are skipped.
[INFO]
[INFO] --- jar:3.4.2:jar (default-jar) @ Ejercicio08-Reservaciones ---
[INFO]
[INFO] --- spring-boot:4.0.6:repackage (repackage) @ Ejercicio08-Reservaciones ---
[INFO] Replacing main artifact C:\Users\luism\Documents\Miguel\Nose\Otro\backend-ejercicio08-main\backend-ejercicio08-main\target\Ejercicio08-
[INFO] The original artifact has been renamed to C:\Users\luism\Documents\Miguel\Nose\Otro\backend-ejercicio08-main\backend-ejercicio08-main\t
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.632 s
[INFO] Finished at: 2026-06-24T19:55:30-06:00
[INFO] -----
```

Figura 6: Compilación exitosa del proyecto.

También se realizó una prueba HTTP autenticada contra el módulo de cuartos. Se creó un cuarto usando credenciales de administrador y posteriormente se consultó la lista de cuartos.

Prueba	Endpoint	Resultado
Crear cuarto	POST /api/v1/cuartos	HTTP 201.
Listar cuartos	GET /api/v1/cuartos	HTTP 200.

```
Prueba HTTP autenticada - Cuartos

POST http://localhost:8081/api/v1/cuartos
Authorization: Basic admin:123456
HTTP 201 Created
{"id":1,"tipo":"Suite Swagger","numero":501,"precio":1200.00,"numeroCamas":2,"disponible":true}

GET http://localhost:8081/api/v1/cuartos
Authorization: Basic admin:123456
HTTP 200 OK
[{"id":1,"tipo":"Suite Swagger","numero":501,"precio":1200.00,"numeroCamas":2,"disponible":true}]
```

Figura 7: Prueba autenticada del módulo de cuartos.

8 Conclusiones

La integración de Swagger/OpenAPI permitió documentar automáticamente el Back End del Sistema de Gestión de Cuartos y Reservaciones. La interfaz Swagger UI facilita la revisión de rutas, métodos HTTP, cuerpos de solicitud, respuestas y módulos disponibles sin revisar manualmente todos los controladores.

El proyecto cumple con los puntos solicitados para la práctica: documentación de cuartos, reservaciones, autenticación y seguridad; acceso público a Swagger UI; generación de OpenAPI JSON; y evidencia de compilación y consumo de endpoints protegidos mediante HTTP Basic.

9 Referencias bibliográficas

Referencias

- [1] Spring, *Spring Boot Reference Documentation*. <https://docs.spring.io/spring-boot/>
- [2] Spring, *Spring Security Reference*. <https://docs.spring.io/spring-security/reference/>
- [3] Springdoc, *OpenAPI 3 Library for Spring Boot*. <https://springdoc.org/>
- [4] OpenAPI Initiative, *OpenAPI Specification*. <https://spec.openapis.org/oas/latest.html>